Name: Syeda Malika Zainab Kazmi

Batch: Pana-101-P011

Course:AI-101

REPORT WRITING

Task 2 — AI Generated Snake Game

- **Deployed Game Link**: https://snake-game-273952851429.asia-southeast1.run.app
- **AI Tool Used**: Google AI Studio (Gemini)

Game Concept Summary

The game produced is an enhanced Snake game built beyond the traditional grid-based format. Following the prompt, it features a nature-inspired gameplay environment with terrain elements, multiple snake species and color options, bug-eating mechanics, obstacles, and creative power-up enhancements. The snake grows dynamically as it eats bugs, and the game ends on collision with itself or obstacles.

➢ Prompt Iterations Used During Development

- **Prompt 1 (Initial — Design & Specification):**

The first prompt was the full design document provided, asking AI Studio to design and outline technical specifications for a unique Snake game. This included the starting page, snake selection screen, gameplay environment with terrain and obstacles, core mechanics (eating bugs, arrow key controls, collision game over), and suggestions for enhancements like power-ups and multiple bug types.

This was a specification-level prompt not asking for code yet, but asking the AI to think through the full game design first.

What worked: AI Studio understood the multi-part structure well and produced a detailed breakdown covering game concept, features, tech stack recommendation (HTML5 Canvas + JavaScript), UI/UX flow, and visual design ideas. The output was organized and covered all the required elements.

What didn't work: The initial specification was somewhat generic the AI defaulted to suggesting common features without injecting much original creativity. The visual ideas were described but not very distinctive.

- **Prompt 2 (Implementation):**

After reviewing the specification, the next prompt asked AI Studio to implement the game using the "Create Apps" feature in AI Studio, converting the specification into a working web-based Snake game using HTML5 Canvas and JavaScript.

What worked: AI Studio's "Create Apps" option was well-suited for this it generated a fully functional single-file web app with the canvas-based game loop, arrow key controls, collision detection, and a start screen with a Start button.

What didn't work: The first implementation was missing snake species selection and the terrain/obstacle visual elements. The environment looked like a plain colored background rather than a meaningful terrain. Power-ups were also not included in the first build.

- **Prompt 3 (Iterative Improvement — Snake Customization):**

Prompt: "Add a snake selection screen before gameplay begins. Let the player choose from at least 3 colors (green, red, blue) and 2 species (cobra, python) with different visual styles. Show a preview of the selected snake on the selection screen."

What worked: AI Studio successfully added a pre-game selection screen with color buttons and species options. The snake preview updated live when selections changed this was a nice touch that worked well on the first try.

What didn't work: The species differences were mostly cosmetic (slight head shape changes) rather than gameplay-affecting. The visual distinction between cobra and python wasn't very strong.

- **Prompt 4 (Iterative Improvement — Environment & Obstacles):**

Prompt: "Add a grass terrain background to the game grid. Place rocks and trees as static obstacles on the map that the snake must avoid. Game over should also trigger if the snake hits an obstacle."

What worked: The terrain background (grass texture using canvas fill patterns) was added successfully. Obstacles were rendered as dark brown rocks scattered around the grid, and collision detection was correctly extended to include them.

What didn't work: The obstacle placement was random each game, which occasionally created unfair starting conditions where obstacles spawned too close to the snake's starting position. This needed an extra fix.

- **Prompt 5 (Bug Fix — Obstacle Spawn):**

Prompt: "Make sure obstacles never spawn within 5 tiles of the snake's starting position. Also ensure food (bugs) never spawn on top of obstacles."

What worked: Both fixes were applied correctly. After this the game felt fair and the food always appeared in reachable locations.

- **Prompt 6 (Enhancement — Power-ups & Scoring):**

Prompt: "Add two power-ups that randomly appear on the map: one that temporarily increases snake speed (lightning bolt icon), and one that briefly makes the snake invincible to self-collision (shield icon). Also add a score display showing current score and a high score that persists during the session."

- **What worked:** Both power-ups were implemented with icons drawn on canvas and timed effects. The scoring system (score per bug eaten, high score tracking) worked correctly. The lightning bolt speed boost was a good gameplay addition.

- **What didn't work:** The invincibility power-up's visual indicator (a glow effect around the snake) was subtle and hard to notice during gameplay. A timer countdown would have been more helpful.

➢ What Worked Well

- Google AI Studio's "Create Apps" feature was excellent for converting the written specification directly into a working single-file HTML/JS app without needing any manual coding setup.
- Iterating with specific, targeted prompts (one feature at a time) produced much cleaner results than trying to ask for everything at once.
- Canvas-based rendering worked smoothly the game ran at a consistent frame rate with no performance issues.
- The snake selection screen with live preview was one of the best features and came out well on the first attempt.
- Collision detection (self, wall, obstacles) was accurate throughout all iterations.

➢ What Did Not Work

- Vague or open-ended prompts produced generic results. Early prompts like "make the game more visually appealing" produced minor color changes rather than meaningful improvements.
- Species-based gameplay differences were hard to get the AI to implement properly it kept interpreting species as purely cosmetic.
- The obstacle random-spawn issue at the start showed that AI-generated code sometimes has edge cases that need a follow-up fix prompt rather than being caught on the first try.
- Sound effects were attempted in one prompt but didn't integrate cleanly in the browser environment, so they were dropped.

➢ How Prompts Were Improved Over Time

The biggest lesson was moving from broad prompts to surgical ones. "Make the game better" gave nothing useful. "Add a grass terrain background using canvas fill patterns and place 8 static rock obstacles with collision detection" gave exactly what was needed. Specificity was everything.

Also switching from "write a game" to using AI Studio's "Create Apps" mode was the right call it kept the output as a deployable single file rather than disconnected code snippets.

- **Challenges Faced**

The main challenge was getting all the features from the specification into the final game without the later prompts breaking earlier ones. Each time a new feature was added, there was a risk of the AI rewriting parts of the existing code incorrectly. The solution was to always describe the new feature in isolation and explicitly say "do not change existing functionality only add this new feature."

Another challenge was deploying the final app to Google Cloud Run. AI Studio itself generates the code, but deployment required packaging it into a container. Following the standard Cloud Run deployment steps for a static HTML app resolved this.